



INDIA.RUNS

Build what next India runs on

Team Name : Match Matrix

Team Leader Name : Karthik Gollapudi

Problem Statement :

Traditional hiring systems rely heavily on keyword matching, often overlooking qualified candidates and ranking unsuitable profiles higher, making candidate discovery inefficient and unreliable.

Solution Overview

- What is your proposed solution?

IntelliHire is an AI-powered candidate ranking system that evaluates job requirements and candidate profiles using career evidence, technical expertise, behavioral signals, and risk-aware scoring to generate explainable Top-N recommendations.

- What differentiates your approach from traditional candidate matching systems?

Unlike traditional keyword-based matching, IntelliHire combines career history, trusted skill validation, behavioral intelligence, and profile quality signals to identify genuinely qualified candidates while penalizing keyword stuffing and weak-fit profiles.

JD Understanding & Candidate Evaluation

- What are the key requirements extracted from the JD?
 - Retrieval, ranking, and semantic search expertise
 - Embeddings and vector database experience
 - Strong Python and production ML engineering skills
 - Ranking evaluation and experimentation mindset
 - Product-focused engineering experience
 - Preferred experience range of 5–9 years

JD Understanding & Candidate Evaluation

- Which candidate signals are most important for determining relevance?

- Current role and title relevance
- Career history and domain experience
- Retrieval, ranking, NLP, and ML skills
- Product-company background
- Years of experience
- Recruiter engagement and activity signals
- Notice period, availability, and profile credibility

- How does your solution evaluate candidate fit beyond keyword matching?

IntelliHire evaluates candidate fit using career evidence, trusted skill validation, behavioral intelligence, product-company experience, and risk-aware scoring, ensuring candidates are ranked based on overall relevance rather than keyword presence alone.

Ranking Methodology

- How does your system retrieve, score, and rank candidates?
 - Candidate profiles are processed to extract career history, skills, experience, behavioral signals, and profile metadata.
 - Multiple relevance scores are computed independently for each candidate.
 - Candidates are ranked based on a weighted composite score after applying risk penalties.

Ranking Methodology

- What models, algorithms, or heuristics are used?
 - Rule-based JD alignment using domain-specific scoring heuristics.
 - Title relevance scoring.
 - Career evidence scoring using retrieval, ranking, and ML-related keywords.
 - Trusted skill validation using proficiency, duration, endorsements, and assessments.
 - Behavioral scoring using recruiter engagement and activity signals.
 - Risk-aware penalties for keyword stuffing, stale profiles, and weak-fit candidates.

Ranking Methodology

- How are multiple candidate signals combined into a final ranking?

Final Score = Weighted Hybrid Score – Risk Penalties

Signals combined:

- Title Relevance (23%)
- Career Evidence (25%)
- Trusted Skills (18%)
- Experience (11%)
- Product Company Experience (10%)
- Location Fit (7%)
- Behavioral Intelligence (6%)

The final score is adjusted using an availability multiplier and risk penalty layer before generating the ranked candidate list.

Explainability & Data Validation

- How are ranking decisions explained?
 - Every recommendation includes a human-readable reasoning summary.
 - Explanations highlight role relevance, career evidence, key skills, behavioral signals, and potential concerns.
 - Recruiters can understand why a candidate was ranked highly or penalized.
- How do you prevent hallucinations or unsupported justifications?
 - All explanations are generated directly from candidate profile data and computed scores
 - No external LLMs or generative models are used during ranking.
 - Recommendations are deterministic, reproducible, and evidence-based. .

Explainability & Data Validation

- How does your solution handle inconsistent, low-quality, or suspicious profiles?
 - Detects and penalizes keyword stuffing.
 - Identifies stale or low-engagement profiles.
 - Flags impossible skill-duration combinations.
 - Applies penalties to services-only career histories and weak retrieval/ranking evidence.
 - Uses risk-aware scoring to reduce the ranking of suspicious profiles.

End-to-End Workflow

- What is the complete workflow from JD input to ranked candidate output?

Step 1: Job Description Analysis

- Extract key technical, experience, and role requirements from the JD.

Step 2: Candidate Profile Processing

- Parse candidate profiles, career history, skills, and behavioral data.

Step 3: Feature Extraction

- Identify title relevance, skills, experience, career evidence, and engagement signals.

Step 4: Candidate Scoring

- Compute Title, Skill, Experience, Career Evidence, Behavioral, and Location scores.

End-to-End Workflow

Step 5: Risk & Quality Assessment

- Detect keyword stuffing, stale profiles, inconsistent claims, and weak-fit candidates.

Step 6: Final Score Generation

- Combine all candidate signals using a weighted hybrid scoring framework.

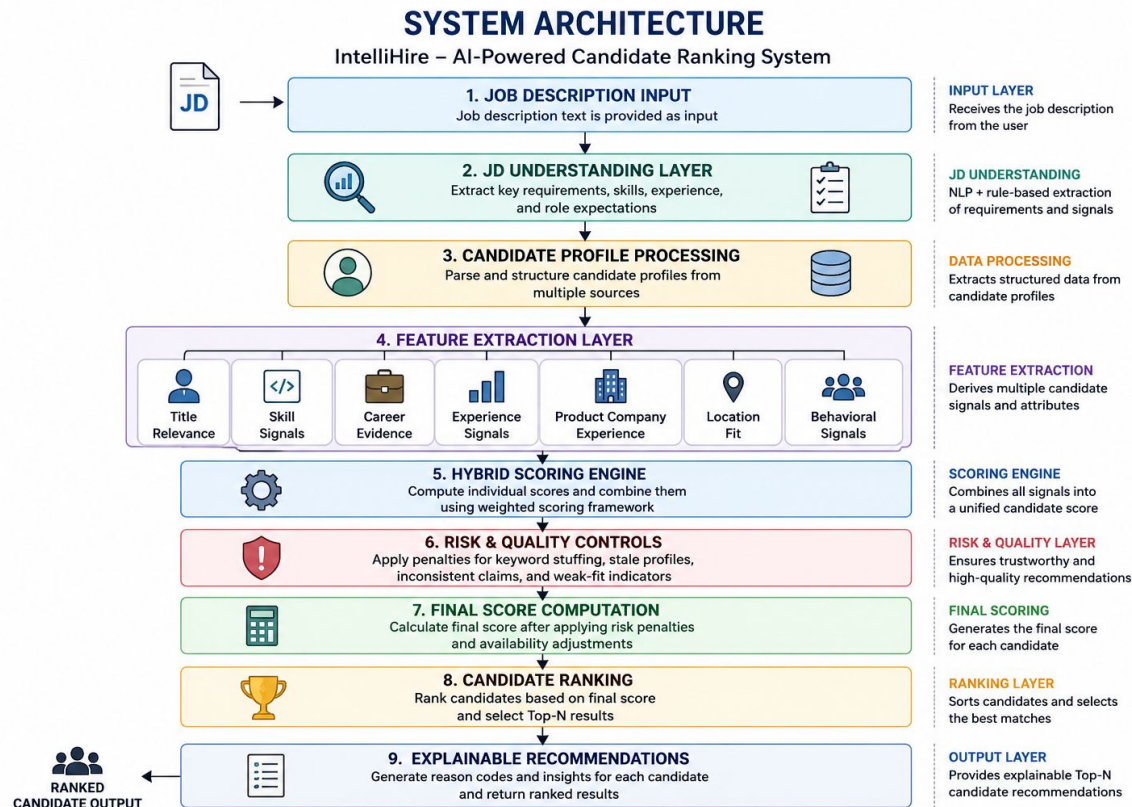
Step 7: Candidate Ranking

- Rank candidates based on final scores and select the Top-N recommendations.

Step 8: Explainable Recommendations

- Generate transparent reasoning for every ranked candidate.

System Architecture



System Architecture Components

JD Understanding Layer

- Extracts role requirements and hiring signals.

Candidate Processing Layer

- Parses candidate profiles, skills, experience, and activity data.

Feature Extraction Layer

- Generates title, skill, career evidence, experience, and behavioral features.

Hybrid Scoring Engine

- Combines multiple candidate signals into a unified score.

Risk & Quality Controls

- Detects keyword stuffing, stale profiles, and inconsistent claims.

Ranking & Recommendation Layer

- Produces explainable Top-N candidate recommendations.

Results & Performance

- What results or insights demonstrate ranking quality?
 - Successfully generated a **Top-100 ranked candidate shortlist** from the complete dataset.
 - Produced **100 unique candidate recommendations** with no duplicates.
 - Generated **explainable reasoning** for every ranked candidate.
 - Combined technical, behavioral, and risk signals to improve candidate relevance beyond keyword matching.
 - Successfully passed the challenge submission validator.

Results & Performance

- How does your solution meet the challenge's runtime and compute constraints?
 - **CPU-only execution** with no GPU requirements.
 - No external APIs, hosted LLMs, or network dependencies.
 - Deterministic and reproducible ranking pipeline.
 - Processes candidates through lightweight heuristic scoring.
 - Streams JSONL candidate data and generates rankings efficiently.
 - Designed to handle **100,000+ candidate profiles** within challenge constraints.

Technologies Used

- What technologies, frameworks, and tools were used and why were they selected for this solution?

Tech Stack

- Python
- JSONL Processing
- Streamlit
- GitHub
- Streamlit Cloud
- CSV

Why These Technologies?

- **Python** → Core ranking engine and scoring pipeline
- **JSONL** → Efficient processing of large candidate datasets
- **Streamlit** → Interactive candidate ranking sandbox
- **GitHub** → Version control and reproducibility
- **Streamlit Cloud** → Public deployment and demonstration
- **CSV** → Challenge-compliant output format

Submission Assets

Links

GitHub

<https://github.com/Karthik-0027/IntelliHire-AI-Powered-Talent-Intelligence-Candidate-Ranking-System>

Sandbox

<https://intellihire-candidate-ranker.streamlit.app/>



INDIA.RUNS

Build what next India runs on

THANK YOU